# 核心语言的操作语义

A *core language* for describing the behavior of a single *AI Core*.

$$\begin{aligned}1\ \text{NPU (rank)} &= 24\ \text{AI Cores}\\ &= 24 \times (\text{AIC} + 2\,\text{AIV}).\end{aligned}$$

Each AIC/AIV core has several pipes, for example PIPE_MTE2, PIPE_V, PIPE_MTE3, ...

This document describes a (slightly modified) core language than the initial version in the `docs/core-language.tex` file, and present an operational semantics.

## 1 参数说明

对单个**硬件配置**来说，可配置的参数如下。右列以 Ascend 910B 实际取值为例。本文定义的良构检查和操作语义均基于上述参数，不固定一代硬件具体参数的取值。

$f$ 是发到数据管道上的运算，由该管道异步完成。$g$ 是标量算子，由程序直接执行，let 立刻绑定结果。二者都可根据 IR 提供的实际运算配置。本文把同步源语（setflag、waitflag、pipebarrier、syncblockset 和 syncblockwait）作为核心语言的原语。

| 参数说明 | | Example: Ascend 910B |
|---|---|---|
| Subcore 种类 | $d \in \mathsf{CoreKind}$ | $\{\mathsf{AIC}, \mathsf{AIV}\}$ |
| 硬件实际的指令流 | $s \in \mathsf{Stream}$ | $\{\mathsf{aic}, \mathsf{aiv}_1, \mathsf{aiv}_2\}$ |
| 指令流到 Subcore 映射 | $\mathsf{kind} : \mathsf{Stream} \to \mathsf{CoreKind}$ | $\mathsf{aic} \mapsto \mathsf{AIC}$<br>$\mathsf{aiv}_1, \mathsf{aiv}_2 \mapsto \mathsf{AIV}$ |
| 直接执行的标量算子 | $g \in \mathsf{Scalar}$ | $\{$`arith`, `get_block_idx`, `get_sub_block_idx`, ...$\}$ |
| 发到管道的运算 | $f \in \mathsf{Operator}_d$ | $\mathsf{Operator}_{\mathsf{AIV}} = \{\mathsf{Vadd}, \mathsf{Load}, \mathsf{Store}, \ldots\}$<br>$\mathsf{Operator}_{\mathsf{AIC}} = \{\mathsf{Mmad}, \mathsf{Fixpipe}, \ldots\}$ |
| SubCore 的地址空间 | $\mathsf{Space}_d$ | $\mathsf{Space}_{\mathsf{AIV}} = \{\mathsf{GM}, \mathsf{UB}\}$<br>$\mathsf{Space}_{\mathsf{AIC}} = \{\mathsf{GM}, \mathsf{L1}, \mathsf{L0A}, \mathsf{L0B}, \mathsf{L0C}\}$ |
| 数据管道 | $\mathsf{Pipe}_d$ | $\mathsf{Pipe}_{\mathsf{AIV}} = \{\mathsf{MTE2}, \mathsf{V}, \mathsf{MTE3}\}$<br>$\mathsf{Pipe}_{\mathsf{AIC}} = \{\mathsf{MTE1}, \mathsf{MTE2}, \mathsf{M}, \mathsf{FIX}\}$ |
| 同步管道名 | $\mathsf{SyncName}$ | $\{\mathsf{S}, \mathsf{ALL}\}$ |

(1)

管道名包括数据管道和同步管道，我们统一使用元变量 $p, q$ 表示如下：

$$p, q \in \mathsf{Pipe} = \Big( \bigcup_{d \in \mathsf{CoreKind}} \mathsf{Pipe}_d \Big) \cup \mathsf{SyncName}. \tag{2}$$

对单个**程序**来说，指定它唤起的指令流 $\kappa \subseteq \mathsf{Stream}$。程序不重新指定种类，一条流的种类是 $\mathsf{kind}(s)$。HIVM 的 `core_ratio` 记录过的比例包括 0:1、1:0、1:1 和 1:2，分别对应

$$\begin{aligned}\kappa_{0:1} &= \{\mathsf{aiv}_1\}, & \kappa_{1:0} &= \{\mathsf{aic}\},\\ \kappa_{1:1} &= \{\mathsf{aic}, \mathsf{aiv}_1\}, & \kappa_{1:2} &= \{\mathsf{aic}, \mathsf{aiv}_1, \mathsf{aiv}_2\}.\end{aligned}$$

纯 AIV 或纯 AIC 的 kernel 使用 $\kappa_{0:1}$ 或 $\kappa_{1:0}$。后文的语法和规则对任意 $\kappa \subseteq \mathsf{Stream}$ 相同。一项程序是 $\kappa$ 中这些指令流的并列，不包含第二个 AI Core。

同一个 AI Core 里有两层同步。setflag、waitflag 与 pipebarrier 只约束一条指令流内部的管道。syncblockset 与 syncblockwait 约束这个 AI Core 内的 AIC 与 AIV。Ascend C 把这一层也叫做核间同步，对应 `CrossCore` 的模式 2。本文保留这两层。多个 AI Core 之间的同步不在本文中，那是 `CrossCore` 的模式 0，以及 block 网格和 rank 之间的通信。

目前的局限：

- 配置只含一个 AI Core。`get_block_idx` 是这个 AI Core 已经给定的整数，各条流读到同一个值。`get_sub_block_idx` 是一条流自己的整数，用来区分 $\kappa$ 里的各条 AIV 流。
- 不处理复杂算子如 `mmadL1` 的内部事件槽、`nd2nz` 和 FFTS 基址。写到 GM 的 `fixpipe` 是 $\mathsf{Operator}_{\mathsf{AIC}}$ 中的一个名字，仍是一条 issue。

## 2 语法

沿用 (1)、(2) 中定义的参数。

$$f \in \mathsf{Operator}_d \quad g \in \mathsf{Scalar} \quad p, q \in \mathsf{Pipe} \quad \text{where } d = \mathsf{kind}(s)(s \in \kappa)$$

$$\begin{aligned}
\tau &::= \mathsf{int} \mid \mathsf{float} \mid \mathsf{bool} \mid \mathsf{event} \mid \mathsf{view} \\
v &::= c \mid x \\
C &::= \mathsf{return}(\bar{v}) \mid \mathsf{end} \\
&\quad \mid \mathsf{let}\ x = \mathsf{scalar}[g](\bar{v})\ \mathsf{in}\ C \\
&\quad \mid \mathsf{issue}[p](f, \bar{v});\ C \\
&\quad \mid \mathsf{setflag}[p, q](v);\ C \mid \mathsf{waitflag}[p, q](v);\ C \\
&\quad \mid \mathsf{pipebarrier}[p];\ C \\
&\quad \mid \mathsf{syncblockset}[p, q](v);\ C \mid \mathsf{syncblockwait}[p, q](v);\ C \\
&\quad \mid \mathsf{if}\ v\ \mathsf{then}\ \{C_1\}\ \mathsf{else}\ \{C_2\} \\
&\quad \mid \mathsf{for}\ x = v_1\ \mathsf{to}\ v_2\ \mathsf{step}\ v_3\ \mathsf{do}\ \{C_1\};\ C_2, \\
P &::= \Big\Vert_{s \in \kappa}\ s\{C_s\}.
\end{aligned}$$

- $P$ 并行执行若干条指令流，$\kappa$ 只有一个元素时，$P$ 就写作 $s\{C\}$。每条指令流内顺序执行。
- $g$ 的例子包括：标量算术运算、`get_block_idx`、`get_sub_block_idx` 和视图构造。$\mathsf{scalar}[g]$ 不进入数据管道。第 3 节的 $\Delta(g)$ 给出它的参数类别和结果类别；let 立刻把 $\mathsf{eval}_g$ 的结果绑定到 $x$。
- $\mathsf{issue}[p](f, \bar{v})$ 不绑定结果。$\bar{v}$ 里的地址是类别为 view（可理解为 `memref` 等地址表示）的操作数，读取和写入都在其中，目的地也是一个操作数。后续使用 $\mathsf{foot}(f, \bar{\nu}) = (R, W)$ 计算视图对应的存储单元集合（表示为 Loc 的一个子集）：

$$\mathsf{Loc} = \mathsf{Space} \times \mathbb{Z}, \quad \mathsf{Space} = \bigcup_d \mathsf{Space}_d, \quad R, W \subseteq \mathsf{Loc}.$$

其中 $R$ 是读取的单元，$W$ 是写入的单元。Loc 只出现在操作语义里。

LITAO:Below are work in progress.

# 3　良构与基本类型检查

本节只检查基本类别、变量作用域和指令/管道的适配，不检查同步是否充分。固定硬件参数与 $\kappa$；对 $s \in \kappa$，记 $D_s = \mathsf{Pipe}_{\mathsf{kind}(s)}$、$O_s = \{t \in \kappa \mid \mathsf{kind}(t) \neq \mathsf{kind}(s)\}$。

$$\Gamma : \mathsf{Var} \rightharpoonup \mathsf{Type}, \qquad \mathsf{lit}(c) = \tau, \qquad \Delta(g) = (\bar{\tau}, \tau), \qquad \Sigma(f, p) = \bar{\tau}.$$

$$\mathsf{Type} = \{\mathsf{int}, \mathsf{float}, \mathsf{bool}, \mathsf{event}, \mathsf{view}\}.$$

同步指令 $a$ 的静态管道条件 $\mathsf{ok}_s(a)$ 如下（事件操作数另行检查）：

| $a$ | $\mathsf{ok}_s(a)$ |
|---|---|
| $\mathsf{setflag}[p,q](v),\ \mathsf{waitflag}[p,q](v)$ | $p, q \in D_s \ \wedge\ p \neq q$ |
| $\mathsf{syncblockset}[p,q](v)$ | $O_s \neq \varnothing \ \wedge\ p \in D_s \ \wedge\ q \in D_s \cup \{\mathsf{S}\} \ \wedge\ p \neq q$ |
| $\mathsf{syncblockwait}[p,q](v)$ | $O_s \neq \varnothing \ \wedge\ p \in \mathsf{Pipe} \ \wedge\ q \in D_s \cup \{\mathsf{S}\} \ \wedge\ p \neq q$ |

$$\frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau}\ \textsc{Var} \qquad \frac{\mathsf{lit}(c) = \tau}{\Gamma \vdash c : \tau}\ \textsc{Lit} \qquad \frac{|\bar{v}| = |\bar{\tau}| \quad \forall i.\ \Gamma \vdash v_i : \tau_i}{\Gamma \vdash \bar{v} : \bar{\tau}}\ \textsc{Tuple}$$

$$\frac{\Delta(g) = (\bar{\tau}, \tau) \quad \Gamma \vdash \bar{v} : \bar{\tau} \quad x \notin \mathrm{dom}(\Gamma) \quad \Gamma, x : \tau \vdash_s C}{\Gamma \vdash_s \mathsf{let}\ x = \mathsf{scalar}[g](\bar{v})\ \mathsf{in}\ C}\ \textsc{Let}$$

$$\frac{p \in D_s \quad f \in \mathsf{Operator}_{\mathsf{kind}(s)} \quad \Sigma(f, p) = \bar{\tau} \quad \Gamma \vdash \bar{v} : \bar{\tau} \quad \Gamma \vdash_s C}{\Gamma \vdash_s \mathsf{issue}[p](f, \bar{v});\ C}\ \textsc{Issue}$$

$$\frac{\mathsf{ok}_s(a) \quad \Gamma \vdash v : \mathsf{event} \quad \Gamma \vdash_s C}{\Gamma \vdash_s a;\ C}\ \textsc{Sync} \qquad \frac{p \in D_s \cup \{\mathsf{ALL}\} \quad \Gamma \vdash_s C}{\Gamma \vdash_s \mathsf{pipebarrier}[p];\ C}\ \textsc{Bar}$$

$$\frac{\Gamma \vdash v : \mathsf{bool} \quad \Gamma \vdash_s C_1 \quad \Gamma \vdash_s C_2}{\Gamma \vdash_s \mathsf{if}\ v\ \mathsf{then}\ \{C_1\}\ \mathsf{else}\ \{C_2\}}\ \textsc{If}$$

$$\frac{\Gamma \vdash v_1, v_2, v_3 : \mathsf{int}, \mathsf{int}, \mathsf{int} \quad x \notin \mathrm{dom}(\Gamma) \quad \Gamma, x : \mathsf{int} \vdash_s C_1 \quad \Gamma \vdash_s C_2}{\Gamma \vdash_s \mathsf{for}\ x = v_1\ \mathsf{to}\ v_2\ \mathsf{step}\ v_3\ \mathsf{do}\ \{C_1\};\ C_2}\ \textsc{For}$$

$$\frac{\Gamma \vdash \bar{v} : \bar{\tau}}{\Gamma \vdash_s \mathsf{return}(\bar{v})}\ \textsc{Return} \qquad \frac{}{\Gamma \vdash_s \mathsf{end}}\ \textsc{End}$$

$$\frac{\forall s \in \kappa.\ \varnothing \vdash_s C_s}{\vdash_\kappa \Big\Vert_{s \in \kappa}\ s\{C_s\}}\ \textsc{Program}$$

Sync 中 $a$ 是上表四种命令之一，$v$ 是它的事件操作数。循环仍采用原有结构限制：循环体不含嵌套 for，每条非 return 路径以 end 结束；程序顶层不使用 end。

**基本类别与检查边界。** Lit 的 lit 由配置给出且单值：普通整数常量的类别是 int，事件标识常量（如 $\varepsilon_0$）的类别是 event，视图描述符常量的类别是 view，不因底层编码为整数而混同。view 表示可寻址区域的描述符，不是该区域里的数据值；具体布局与足迹仍由已有的 foot 解释。Tuple 同时检查参数个数与逐项类别。例如配置可以声明 $\Delta(\mathsf{makeView}) = (\mathsf{int}^3, \mathsf{view})$，$\Sigma(\mathsf{Copy}, \mathsf{MTE2}) = (\mathsf{view}, \mathsf{view})$，其中 $\mathsf{int}^3$ 表示三个整数参数；实际算子签名以所选 IR 实例为准。因此在要求 view 的位置传整数、在同步位置传 view、或使用未绑定变量都会被拒绝。

Let 仅通过 $\Delta$ 绑定标量结果；Issue 仅检查 $\Sigma$ 与可用管道，不引入数据结果变量。Sync 检查 event 类别与管道条件，暂保留原稿的块同步端点约束；静态可用性不等于实际 set/wait 已配对。**待讨论：** 原稿对块置位的 $q$ 按发送流检查，而后文把它用作接收流的等待管道；块等待的 $p$ 也未限定为某个发送流的数据管道。确定通知/接收语义后，应一起收紧这两项，本轮不默改这一设计。For 的循环变量只在循环体内可见；Return 不规定返回签名，但每个返回操作数必须可定型。Program 中各条流的变量环境互不共享。

这是一项可按语法递归执行的基本 type check，不是同步正确性证明。它不要求 flag 已置位、存储已初始化或 view 互不重叠，也不证明地址边界、实际内存空间可访问性、循环步长非零、无竞争或无死锁；这些属于后续动态语义、配置约束或验证目标。缺少同步的程序仍可通过基本类型检查，从而让语义暴露其错误执行。

# 4 状态与参数

一个指令流 $s \in \kappa$ 的队列 $Q_s$ 把 $\mathsf{Pipe}_{\mathsf{kind}(s)}$ 中每条数据管道映到有限序列。队列项是

$$\begin{aligned}\iota ::= \ & \mathsf{op}(f, R, W) \mid \mathsf{set}(q, \varepsilon) \mid \mathsf{wait}(p, \varepsilon) \\ & \mid \mathsf{bar} \mid \mathsf{bset}(q, \varepsilon) \mid \mathsf{bwait}(p, \varepsilon).\end{aligned}$$

$R, W$ 是有限的 Loc 集合。存储 $\mu$ 把 Loc 部分映到 Uninit 或值。每条指令流有自己的核内 flag 表 $\varphi_s$。块同步不使用这张表。共用的块同步状态是

$$\beta_t(p, q, \varepsilon) \in \{0, 1\},$$

其中 $t$ 是被通知的指令流，初值为 0。一次置位按下面的规则写入对方种类的每一条流；一次等待只清除本流的那一位。

配置含共享的 $\mu$ 与 $\beta$，以及每条 $s \in \kappa$ 的 $\varphi_s$、$Q_s$、环境 $\rho_s$ 和控制 $C_s$。$\rho_s$ 把变量映到值，常量映到自身。实例化提供三个部分。$\mathsf{eval}_g$ 在该指令流上、与 $\Delta(g)$ 相符的值上得到结果；`get_block_idx` 与 `get_sub_block_idx` 直接返回配置里给定的整数。$\mathsf{foot}(f, \bar{\nu}) = (R, W)$ 给出足迹，并且满足 $R \cup W \subseteq \mathsf{Space}_{\mathsf{kind}(s)} \times \mathbb{Z}$。关系 $F(f, \mu|_R, \mu')$ 成立时，$\mu'$ 与 $\mu$ 只在 $W$ 上不同。$F$ 可以有多个结果。这里的 $f$ 属于 $\mathsf{Operator}_{\mathsf{kind}(s)}$。

队列中位于等待项或屏障项之后的项尚未打开。op 在自身已经打开时可以完成，不要求位于队首。set、bset 与 bar 只有在前面的项都已离开队列时才能完成。

# 5 小步规则

一步只前进一条指令流，可以改写共享的 $\mu$ 与 $\beta$。下面省去未改变的分量。控制规则使用当前指令流的环境，规则中记作 $\rho$。

标量求值后继续。if 按布尔值进入一支。return 没有控制后继；这条指令流的队列仍可完成。

$$\frac{\bar{\nu} = \rho(\bar{v}) \quad \mathsf{eval}_g(\bar{\nu}) = \nu}{\mathsf{let}\ x = \mathsf{scalar}[g](\bar{v})\ \mathsf{in}\ C \longrightarrow \rho[x \mapsto \nu],\ C}$$

发射把一项加到目标管道的队尾，控制立即继续。足迹在发射时确定。foot 或 eval 没有结果时，相应的控制规则不启用。

$$\frac{\mathsf{foot}(f, \rho(\bar{v})) = (R, W)}{Q(p),\ \mathsf{issue}[p](f, \bar{v});\ C \longrightarrow Q(p){\cdot}\mathsf{op}(f, R, W),\ C}$$

$\mathsf{setflag}[p,q](v)$ 把 $\mathsf{set}(q,\varepsilon)$ 放入 $p$。$\mathsf{waitflag}[p,q](v)$ 把 $\mathsf{wait}(p,\varepsilon)$ 放入 $q$，不挡住发射流。这里 $\varepsilon = \rho(v)$。$\mathsf{pipebarrier}[p]$ 在 $p$ 放入 bar；$p = \mathsf{ALL}$ 时在本指令流每条数据管道放入 bar。$\mathsf{syncblockset}[p,q](v)$ 把 $\mathsf{bset}(q,\varepsilon)$ 放入 $p$。等待管道不是 S 的 syncblockwait 把 bwait 放入本指令流的等待管道。$\varepsilon$ 同样取 $\rho(v)$。

记 $\mathsf{Opp}(s) = \{t \in \kappa \mid \mathsf{kind}(t) \neq \mathsf{kind}(s)\}$。$\mathsf{bset}(q,\varepsilon)$ 位于 $s$ 的管道 $p$ 的队首，并且每个 $t \in \mathsf{Opp}(s)$ 都有 $\beta_t(p,q,\varepsilon) = 0$ 时，该项离开队列，并把这些 $\beta_t(p,q,\varepsilon)$ 写成 1。其中某一位仍为 1 时，这条规则不启用。因此一次 AIC 上的置位会通知每一条 AIV 流，两条 AIV 流各自等待、各自清除，互不影响。一次 AIV 上的置位只通知 AIC 流。同一种类的指令流不互相通知。同一事件若由两条 AIV 流各置位一次，后完成的那一次会因 AIC 上的位仍为 1 而不启用。

等待管道为 S 时，块等待留在控制位置，该指令流不能再发射。本流的令牌为 1 时只清除这一位并继续。其他指令流，以及本流已经入队的完成，不受这条约制。

$$\frac{\varepsilon = \rho(v) \quad \beta_s(p, \mathsf{S}, \varepsilon) = 1}{\beta,\ \mathsf{syncblockwait}[p, \mathsf{S}](v);\ C \longrightarrow \beta[(s, p, \mathsf{S}, \varepsilon) \mapsto 0],\ C}$$

数据项完成时读取当时的 $\mu|_R$。其中可以有 Uninit，也可以有更早写入的值。规则不检查其他在途指令。

$$\frac{\mathsf{op}(f, R, W) \text{ 已打开} \quad F(f, \mu|_R, \mu')}{\mu,\ Q \longrightarrow \mu',\ Q \setminus \mathsf{op}(f, R, W)}$$

$\mathsf{set}(q,\varepsilon)$ 位于 $p$ 的队首且 $\varphi_s(p,q,\varepsilon) = 0$ 时，该项离开队列，令牌改为 1。$\varphi_s$ 已为 1 时这条规则不启用。$\mathsf{wait}(p,\varepsilon)$ 在 $q$ 上已经打开且令牌为 1 时离开队列，令牌改回 0；令牌为 0 时不启用。打开条件使排在等待后面的同管道项不能先完成。bar 只在位于队首时离开。$\mathsf{bwait}(p,\varepsilon)$ 在本指令流上已经打开且 $\beta_s(p,q,\varepsilon) = 1$ 时离开队列，并且只把 $\beta_s(p,q,\varepsilon)$ 改回 0。核内表 $\varphi_s$ 与块同步状态 $\beta$ 不混用。下一次同一 $(p,q,\varepsilon)$ 的置位要等上次通知到的每一条流都已把自己的那一位清回 0。某一条被通知的流没有执行对应的等待时，这一位保持为 1，下一次置位不启用。

循环按整数范围展开。$k_3 > 0$ 且 $k_1 < k_2$ 时，把循环体中的 end 换成下一次迭代，并绑定本轮的 $x$。范围为空时进入循环后的命令。$C \triangleright D$ 表示这个替换；return 保持不变，if 的两支分别替换。

$$\frac{\rho(v_i) = k_i \quad k_3 > 0 \quad k_1 < k_2}{\mathsf{for}\ x = v_1\ \mathsf{to}\ v_2\ \mathsf{step}\ v_3\ \mathsf{do}\ \{C_1\};\ C_2 \longrightarrow \mathsf{let}\ x = k_1\ \mathsf{in}\ C_1 \triangleright C_+}$$

其中 $C_+$ 是下界改为 $k_1 + k_3$ 的同一个 for。负步长与范围为空的规则方向相反，写法相同。

这些规则不产生名为竞争或死锁的结论。令牌仍为 0 的等待没有完成步骤；该指令流上排在这次等待后面的同管道项保持关闭。其他指令流可以继续。

# 6 与 HIVM 的对应

本节的 $\Delta$、$\Sigma$、$F$ 和 foot 只覆盖两个例子用到的运算名。Copy、MulAdd 属于 $\mathsf{Operator}_{\mathsf{AIV}}$，Fixpipe 属于 $\mathsf{Operator}_{\mathsf{AIC}}$。复制关系 $F_{\mathrm{copy}}$ 按足迹中的对应单元传递当前内容，包括 Uninit。

| HIVM | 核心形式 | 实例 |
|---|---|---|
| `arith`、视图构造、`get_block_idx` | $\mathsf{scalar}[g]$ | $\Delta$ 与 eval |
| `load` | $\mathsf{issue}[\mathsf{MTE2}](\mathsf{Copy},\dots)$ | $F_{\mathrm{copy}}$，GM 到 UB |
| `store` | $\mathsf{issue}[\mathsf{MTE3}](\mathsf{Copy},\dots)$ | $F_{\mathrm{copy}}$，UB 到 GM |
| `fixpipe` | $\mathsf{issue}[\mathsf{FIX}](\mathsf{Fixpipe},\dots)$ | $F_{\mathrm{copy}}$，L0C 到 GM |
| 向量函数 `vf_3` | $\mathsf{issue}[\mathsf{V}](\mathsf{MulAdd},\dots)$ | 三块输入都是值时按乘加写入目的地；含 Uninit 时目的地记为 Uninit |
| `set_flag`、`wait_flag`、`pipe_barrier` | 同名核内命令 | 本指令流的 $\varphi_s$ 与队列 |
| `sync_block_set`、`sync_block_wait` | syncblockset、syncblockwait | 按被通知指令流分开的 $\beta_t$ |

### 6.1 一个 AIV 上的乘加

`debug-memory.mlir` 中的 `@mul_add_kernel` 使用 $\kappa_{0:1}=\{\mathsf{aiv}_1\}$，没有 `sync_block`。三次装载分别使用事件 $0,1,2$。下面写出事件 0 的一次装载，以及随后的乘加和两次写回；另外两次装载只更换事件和视图。源程序里由运行时决定的 `scf.if` 在进入该项之前取定分支，`set_ctrl` 不进入语言。动态长度已经算进视图。

$$
\begin{aligned}
&\mathsf{aiv}_1\{\mathsf{setflag}[\mathsf{V},\mathsf{MTE2}](\varepsilon_0);\\
&\quad \mathsf{waitflag}[\mathsf{V},\mathsf{MTE2}](\varepsilon_0);\mathsf{issue}[\mathsf{MTE2}](\mathsf{Copy},gm_0,u_0);\\
&\quad \mathsf{setflag}[\mathsf{MTE2},\mathsf{V}](\varepsilon_0);\mathsf{waitflag}[\mathsf{MTE2},\mathsf{V}](\varepsilon_0);\\
&\quad \mathsf{issue}[\mathsf{V}](\mathsf{MulAdd},u_0,u_1,u_2,u_3);\\
&\quad \mathsf{setflag}[\mathsf{V},\mathsf{MTE3}](\varepsilon_0);\mathsf{waitflag}[\mathsf{V},\mathsf{MTE3}](\varepsilon_0);\\
&\quad \mathsf{issue}[\mathsf{MTE3}](\mathsf{Copy},u_3,gm_3);\mathsf{issue}[\mathsf{MTE3}](\mathsf{Copy},u_3,gm_4);\\
&\quad \mathsf{pipebarrier}[\mathsf{ALL}];\mathsf{return}()\}.
\end{aligned}
$$

`vf_3` 内部的 AVE 装载、乘法和加法都包在同一次 MulAdd 里。去掉 $\mathsf{MTE2}\to\mathsf{V}$ 这一对后，乘加项仍可在装载完成前打开并完成。它读到的是当时 UB 中的值。若那些单元仍是装载前的内容，写回 Global Memory 的就是这些内容的乘加结果。这是一次普通的完成步骤。

### 6.2 AIC 与 AIV 的一次交接

这次交接使用 $\kappa_{1:1}=\{\mathsf{aic},\mathsf{aiv}_1\}$。源文件是 `attention_hivm_lowered.mlir`。指令流 aic 对应函数 `_attn_fwd_mix_aic`。它用 `fixpipe` 写入 workspace，再执行 `sync_block_set`：核类型为 `CUBE`，管道为 `PIPE_FIX` 与 `PIPE_S`，flag 为 0。指令流 $\mathsf{aiv}_1$ 对应函数 `_attn_fwd_mix_aiv`。它先执行管道和 flag 相同、核类型为 `VECTOR` 的 `sync_block_wait`，再把 workspace 装入 UB。源指令中的核类型由 kind 给出，不另写进项。下面只保留这一次交接，不展开 `mmadL1`、`nd2nz` 和 FFTS 基址。L0C 中的值是 `fixpipe` 的输入。

$$
\begin{aligned}
&\mathsf{aic}\{\mathsf{issue}[\mathsf{FIX}](\mathsf{Fixpipe},\ell,w);\mathsf{syncblockset}[\mathsf{FIX},\mathsf{S}](\varepsilon_0);\mathsf{return}()\}\\
&\quad\|\ \mathsf{aiv}_1\{\mathsf{syncblockwait}[\mathsf{FIX},\mathsf{S}](\varepsilon_0);\mathsf{issue}[\mathsf{MTE2}](\mathsf{Copy},w,u);\mathsf{return}()\}.
\end{aligned}
$$

syncblockset 要等同一条 `PIPE_FIX` 队列里排在它前面的 `fixpipe` 离开，并且 $\mathsf{Opp}(\mathsf{aic})$ 中每条流的对应位都为 0，才把这些位写成 1。在 $\kappa_{1:1}$ 里被通知的只有 $\mathsf{aiv}_1$。它的等待管道是 S，因此在 $\beta_{\mathsf{aiv}_1}(\mathsf{FIX},\mathsf{S},\varepsilon_0)$ 变为 1 之前不能发射后面的 `load`。等待完成时只清除 $\mathsf{aiv}_1$ 这一位。换成 $\kappa_{1:2}$

时，同一次 AIC 置位同时把 $\mathsf{aiv}_1$ 与 $\mathsf{aiv}_2$ 的对应位写成 1，两条 AIV 流各自清除自己的位。去掉 syncblockwait 后，$\mathsf{aiv}_1$ 可以先发射并完成装载，读到 `fixpipe` 写入之前 workspace 里的内容。

# A 指令队列调查

本附录记录 Ascend 910B 这一类分离核上的指令队列，不作为 (1) 的硬件参数。深度取自 Ascend C 文档对 NPU 架构 2201 给出的指令队列深度，数据通路取自同一文档的搬运单元说明。一个 AI Core 含一套 AIC 队列，以及 $\mathsf{aiv}_1$、$\mathsf{aiv}_2$ 各一套 AIV 队列。深度为 0 表示该核没有这条执行单元。

| 队列 | 执行单元 | AIC 上的数据通路 | AIC 深度 | AIV 上的数据通路 | AIV 深度 |
|---|---|---|---|---|---|
| MTE2 | 搬入 DMA | GM → L1，也可 GM → L0A/L0B | 32 | GM → UB | 16 |
| MTE1 | L1 往 Cube 入口搬 | L1 → L0A、L1 → L0B、L1 → BiasTable | 32 | 无此队列 | 0 |
| M | Cube | 读 L0A、L0B，读写 L0C | 16 | 无此队列 | 0 |
| FIX | FixPipe | L0C → GM、L0C → L1 | 32 | 无此队列 | 0 |
| MTE3 | 搬出 DMA | L1 → GM | 32 | UB → GM | 16 |
| V | Vector | 无此队列 | 0 | 读 UB，写 UB | 64 |

Scalar 是每条流的发行者，上述深度表没有给出 Scalar 的队列深度。ALL 不是一条指令队列。